CS220 Lab#8
32-bit MIPS Processor
(Part-2: Branch, Jump, and Comparison)

Mainak Chaudhuri
Indian Institute of Technology Kanpur

Sketch

- In this lab, you will incorporate conditional branch, jump, and comparison instructions in the three-cycle MIPS processor design from the previous lab
- Assignment#1: Lift the limit on the number of prints
  - Carries three marks
- Assignment#2: Incorporate the branch, jump, and comparison instructions
  - Carries four marks

Assignment#1

- Programs with arbitrarily large number of prints should run correctly
  - Processor continues to have four I/O registers
  - The fifth print system call should stall the processor and generate an *io_stall* signal as output from the Processor module
  - The *io_stall* signal output should be relayed and output by the Computer module
  - The environment, on seeing this *io_stall* signal, reads all the I/O registers and notifies the Computer module through a new input (say, *copied_io_regs*) that the I/O registers have been copied

Assignment#1

- Programs with arbitrarily large number of prints should run correctly
  - The processor sets *io_stall* to zero on seeing the *copied_io_regs* input and comes out of stall when *copied_io_regs* becomes zero
    - The processor starts executing the print syscall on which it stalled
  - When a running program ends and the *done* output becomes one, the environment needs to know the number of residual prints to be captured i.e., which I/O registers to read
    - In a program with ten prints, two episodes of four prints each and two more prints when *done* is one

Assignment#1

- Programs with arbitrarily large number of prints should run correctly
  - Outputting the *io_reg_index* is enough to find out which I/O registers to print after *done* becomes one
- Implement the necessary logic in the Processor module, update the input/output ports of the Processor and Computer modules

Assignment#1

- Simulate the design using a testbench that implements the translation of the following

```
int x = –10;
x++;
print x;
x = x << 10;
print x;
x = x >> 3;
print x;
x--;
print x;
x = x & (x – 1);
print x;
x = x | (x + 1);
print x;
exit();
```

Assignment#1

- Synthesize and implement the design
- Generate bit stream, export the hardware
- Write a Vitis IDE C program to run the C program in the previous slide on the design
  - The structure of the Vitis C program is shown in the next few slides
  - Pay attention to how I/O register contents are captured and printed
- Show the output of your Vitis C program to the TAs to receive marks
  - The printed values must match the expected output

Assignment#1

- Structure of Vitis C program

```
unsigned reset=1, ins_addr, ins, done_storing=0, done,
total_cycles, proc_cycles;
int out[4];
unsigned flush_io_regs, done_copying_io_regs=0,
io_regs_index, print_out_count=1;
Xil_Out32(base_addr, reset);
Xil_Out32(base_addr+3, done_storing);
Xil_Out32(base_addr+4, done_copying_io_regs);
reset=0;
Xil_Out32(base_addr, reset);
// Input the translated instruction sequence
// See previous lab's Vitis C program for an example
```

Assignment#1

- Structure of Vitis C program

```
done_storing = 1;
Xil_Out32(base_addr+3, done_storing);
while (1) {
   flush_io_regs = 0;
   done = 0;
   while (!flush_io_regs && !done) {
      flush_io_regs = Xil_In32(base_addr+12);
      done = Xil_In32(base_addr+5);
   }
```

Assignment#1

- Structure of Vitis C program

```
if (flush_io_regs) {
   out[0] = Xil_In32(base_addr+6);
   out[1] = Xil_In32(base_addr+7);
   out[2] = Xil_In32(base_addr+8);
   out[3] = Xil_In32(base_addr+9);
   done_copying_io_regs = 1;
   Xil_Out32(base_addr+4, done_copying_io_regs);
   done_copying_io_regs = 0;
   Xil_Out32(base_addr+4, done_copying_io_regs);
   for (int i=0; i<4; i++) {
      xil_printf("out%d=%d\n\r", print_out_count,
out[i]);
      print_out_count++;
   }
}
```

Assignment#1

- Structure of Vitis C program

```
        if (done) {
            io_regs_index = Xil_In32(base_addr+13);
            int i=6;
            while (io_regs_index > 0) {
                out[0] = Xil_In32(base_addr+i);
                xil_printf("out%d=%d\n\r", print_out_count, out[0]);
                io_regs_index--;
                print_out_count++;
                i++;
            }
            break;
        }
    } // while(1) loop ends
    total_cycles = Xil_In32(base_addr+10);
    proc_cycles = Xil_In32(base_addr+11);
    xil_printf("Total cycles: %u, computation cycles: %u\n\r",
total_cycles, proc_cycles);
    printf("Processor cycles per instruction (CPI): %f\n\r",
((float)proc_cycles)/(ins_addr+1));
```

Assignment#2

- Incorporate the following instructions in the ALU module
  - Opcode=0 (func values in parentheses): jr (0x8), jalr (0x9), slt (0x2a), sltu (0x2b)
    - R format encoding
    - jr uses src1
    - jalr uses src1 and has rd set to 5'd31
    - slt compares $signed(src1) < $signed(src2)
    - sltu compares src1 < src2
    - src1 is regfile[rs] and src2 is regfile[rt]
    - It is a common practice to encode the unused fields as zero

opcode (6)	rs (5)	rt (5)	rd (5)	sh amt (5)	function (6)

Assignment#2

- Incorporate the following instructions in the ALU module
  - Opcodes in parentheses: bltz (0x1), bgez (0x1), beq (0x4), bne (0x5), blez (0x6), bgtz (0x7), slti (0xa), sltiu (0xb)
    - I format encoding
    - bltz compares $signed(src1) < $signed(32'b0), rt=0
    - bgez compares $signed(src1) >= $signed(32'b0), rt=1; bltz and bgez are distinguished by the rt field
    - beq compares (src1 == src2)
    - bne compares (src1 != src2)
    - blez compares $signed(src1) <= $signed(32'b0)
    - bgtz compares $signed(src1) > $signed(32'b0)

Assignment#2

- Incorporate the following instructions in the ALU module
  - Opcodes in parentheses: bltz (0x1), bgez (0x1), beq (0x4), bne (0x5), blez (0x6), bgtz (0x7), slti (0xa), sltiu (0xb)
    - I format encoding
    - slti compares $signed(src1) < $signed(sign-extended immediate)
    - sltiu compares src1 < sign-extended immediate
    - src1 is regfile[rs] and src2 is regfile[rt]
    - It is a common practice to encode the unused fields as zero

Assignment#2

- Incorporate the following instructions in the ALU module
  - Opcodes in parentheses: j (0x2), jal (0x3)
    - J format encoding
    - In this implementation, we will use the jump_target field in the instruction as the actual target
      - Simplified design, deviates from MIPS ISA specification
    - The jal instruction has an implicit destination register number 31 to store the return address

Assignment#2

- Additional ALU inputs
  - pc (program counter of the current instruction): used in the target computation of conditional branches
  - branch_offset: sign-extended immediate for conditional branches or jump_target for j, jal
  - rt: used to distinguish between bltz and bgez

Assignment#2

- Additional ALU outputs
  - Execution of branch and jump instructions produce two outputs
    - The branch/jump target and whether the target should be used to modify next_pc (referred to as branch taken e.g., beq is taken if src1 is equal to src2)
      - Branch target = pc + branch_offset for conditional branches
      - Branch target = jump_target for j and jal
      - Branch target = src1 for jr and jalr
      - Branch taken is 1 for j, jal, jr, jalr
      - Branch taken depends on condition evaluation for conditional branches e.g., branch taken = (src1 == src2) ? 1'b1 : 1'b0 for beq

Assignment#2

- Additional ALU outputs
  - The already existing output *dest* should be used to convey the branch/jump target for all branch/jump instructions except jal and jalr
    - For jal and jalr, the *dest* output should be pc+1
    - For jal and jalr, the jump target is not computed in ALU; instead, the processor sets the branch target for these instructions to the jump_target field of the instruction (for jal) or src1 (for jalr)
  - A new ALU output should be introduced to convey the branch taken value
- Remember to follow the rules of a combinational always block when updating your ALU module

Assignment#2

- The *dest_valid* output of ALU should be set to zero for all conditional branch and j, jr instructions
- The *dest_valid* output should be set to 1 for jal and jalr
  - These instructions write pc+1 to regfile[31]
  - Recall that *dest_valid* is used to compute write enable for the register file
- Update the Processor module to incorporate the new logic for *next_pc* computation and to send the additional inputs to ALU

Assignment#2

- Simulate your design to show that it produces the correct outputs when the following C code is translated and run on the design

```
int i, x=10, N=30;
for (i=0; i<N; i++) {
   x += i;
   print x;
}
exit();
```

Assignment#2

- Synthesize, implement, generate bit stream, export hardware, and write a Vitis IDE C program to run the program of the previous slide on your design
  - Follow the structure of the Vitis C program as used in Assignment#1
  - The outputs should match with the expected results
  - Show your outputs to the TAs
- To get marks, you must translate and run the program given in the next slide on your design through Vitis and show expected outputs

Assignment#2

- Final program:
  - No need to save $ra before calling f

```
int i, x=10, N=30;
for (i=0; i<N; i++) {
   if ((i & 0x1) == 0) {
      x += f(x, i);
      print x;
   }
   else {
      x –= f(x, i);
      print x;
   }
}
exit ();
int f (int a, int b) { return a+b; }
```